

MODULE ALGORITHMIQUE ET STRUCTURES DE DONNÉES

PILES

©B. DERDOURI

CM- PILES Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

1

PLAN

- Introduction
- Exemple & Définition
- Intérêts & application
- Opérations de bases & Représentation
- Exemple : Evaluation Expression Post-Fixée
- Implémentation contiguë & opérations Piles
- Implémentation chaînée & opérations Piles

©B. DERDOURI

CM Piles Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

2

PILES

Introduction

- Une pile est une structure de liste particulière. Elle ne sert pas à garder de façon plus ou moins définitive des données.
- On s'intéresse plutôt à la suite des états de la pile
- et on applique le principe suivant : dernier arrivé, premier servi (LIFO : Last In First Out).

EXEMPLES

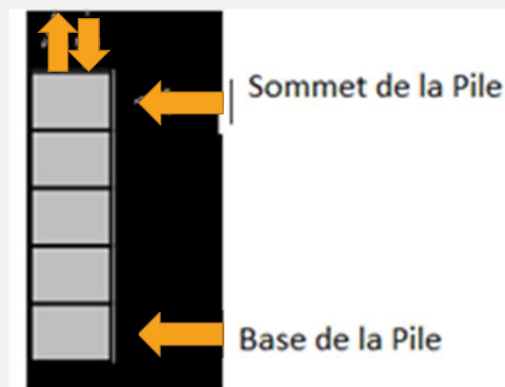
- Pile d'assiettes : on peut ajouter et enlever des assiettes au sommet de la pile.
- Toute insertion ou retrait d'une assiette en milieu d'une pile est une opération qui comporte des risques (surtout si la pile est haute !).



DÉFINITION

- Une pile est une liste linéaire dont les éléments doivent entrer et sortir conformément à la stratégie LIFO.
- Le premier élément de la pile est appelé sommet de la pile
- Le dernier élément de la pile est appelé base de la pile.
- D'après la définition, Les mises à jour de chaque pile se font à partir du sommet de la pile.
- Une pile est en général schématisée par la suite chronologique de ses états.

DÉFINITION



INTÉRÊTS

- La structure de pile est utilisée pour sauvegarder temporairement des informations en respectant leur ordre d'entrée, et les réutiliser en ordre inverse.

APPLICATIONS

- Les applications utilisant des piles sont nombreuses :
 - Evaluation des fonctions récursives
 - Evaluations des expressions
 - Parcours d'arborescences
 - Etc

OPÉRATIONS DE BASE

- **estVidePile** : teste si une pile donnée est vide ou pas
- **estPleinPile** : teste si une pile donnée est pleine ou pas
- **initPile** : initialise une pile
- **sommetPile** : retourne l'élément au sommet de la pile sans que la pile soit modifiée
- **empiler** : ajout un nouvel élément au sommet d'une pile
- **dépiler** : supprime l'élément au sommet d'une pile

REPRÉSENTATIONS

- Une pile est une liste linéaire, elle peut donc être représentée de deux manières :
 - contiguë
 - chaînée

REMARQUE

- Les différentes primitives (opérations) ne tiennent pas compte de la représentation.
- Seul le corps des primitives, inaccessible à l'utilisateur, dépend de cette représentation.

EXEMPLE

- Evaluation d'une expression Post-Fixée

REPRÉSENTATION CONTIGUË D'UNE PILE

LANGAGE ALGORITHMIQUE

```

Type TElement =
Type Pile = Enregistrement (=Structure )

    hauteurMax : Entier

    hauteur : Entier

    tab : Tableau de TElement

Fin
  
```

LANGAGE C

```

Typedef ... Telement
Typedef struct{

    int hauteurMax ;

    int hauteur ;

    TElement *tab ;

}Pile ;
  
```

OPÉRATIONS

- **Fonction allocMemPile : Entier → Pile**
- **Fonction allocMemPile(hMax) → Pile**
- **P.F hMax : Entier (E)**
- **Début**
 - $p.\text{hauteurMax} \leftarrow h\text{max}$
 - $p.\text{tab} \leftarrow \text{allocMemTab}(h\text{Max})$
 - renvoyer p
- **FIN**
- **V.I p : Pile**

OPÉRATIONS

- **Fonction libMemPile : Pile → Pile**
- **procédure libMemPile(p)**
- **P.F p : Pile (E/S)**
- **Début**
- libMemTab(p.tab)
- **Fin**

OPÉRATIONS

- **Fonction initPile : Pile → Pile**
- **Fonction initPile(p)→ Pile**
- **P.F p : Pile (E)**
- **Début**
- p.hauteur ← 0
- renvoyer p
- **Fin**

OPÉRATIONS

- **Fonction hautMaxPile : Pile $\rightarrow$ Entier**
- **Fonction hautMaxPile(p) $\rightarrow$ Entier**
- P.F p : Pile (E)
- **Début**
- Renvoyer p.hauteurMax
- **Fin**

OPÉRATIONS

- **Fonction hautPile : Pile $\rightarrow$ Entier**
- **Fonction hautPile(p) $\rightarrow$ Entier**
- P.F p : Pile (E)
- **Début**
- Renvoyer p.hauteur
- **Fin**

OPÉRATIONS

- **Fonction estVidePile : Pile $\rightarrow$ Booléenne**
- **Fonction estVidePile(p) $\rightarrow$ Booléenne**
- P.F p : Pile (E)
- **Debut**
- Renvoyer **hauteurPile(p)=0**
- **Fin**

OPÉRATIONS

- **Fonction estPleinPile : Pile $\rightarrow$ Booléenne**
- **Fonction estPleinPile(p) $\rightarrow$ Booléenne**
- P.F p : Pile (E)
- **Debut**
- Renvoyer **hauteurPile(p) = hauteurMaxPile(p)**
- **Fin**

OPÉRATIONS

- **Fonction sommetPile : Pile $\rightarrow$ TElement**
- **PréCondition : non estVidePile(p)**
- **Fonction sommetPile(p) $\rightarrow$ TElement**
- P.F p : Pile (E)
- **Début**
- Renvoyer p.tab[hauteurPile(p)]
- **FIN**

©B. DERDOURI

CM Piles Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

21

OPÉRATIONS

- **Fonction empiler : Pile $\rightarrow$ TElement**
- **PréCondition : non estPleinPile(p)**
- **Procédure empiler(elt, p)**
- P.F elt : TElement (E) p : Pile (E/S)
- **Début**
- p.hauteur $\leftarrow$ p.hauteur + 1
- p.tab[p.hauteur] $\leftarrow$ elt
- **FIN**

©B. DERDOURI

CM Piles Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

22

OPÉRATIONS

- **Fonction empiler : Pile $\rightarrow$ TElement**
- **PréCondition : non estPleinPile(p)**
- **fonction empiler(elt, p) $\rightarrow$ Pile**
- P.F elt : TElement (E) p : Pile (E)
- **Début**
 - $p.hauteur \leftarrow p.hauteur + 1$
 - $p.tab[p.hauteur] \leftarrow elt$
 - Renvoyer p
- **FIN**

OPÉRATIONS

- **Fonction depiler : Pile $\rightarrow$ Pile**
- **PréCondition : non estVidePile(p)**
- **Procédure depiler(p)**
- P.F p : Pile (E/S)
- **Début**
 - $p.hauteur \leftarrow p.hauteur - 1$
- **FIN**

OPÉRATIONS

- **Fonction depiler : Pile $\rightarrow$ Pile**
- **PréCondition : non estVidePile(p)**
- **Fonction depiler(p) $\rightarrow$ Pile**
- P.F p : Pile (E)
- **Début**
 - $p.\text{hauteur} \leftarrow p.\text{hauteur} - 1$
 - renvoyer p
- **FIN**

REPRÉSENTATION CHAÎNÉE D'UNE PILE

```

Typedef ... TElement;
Typedef struct cellule{
    TElement donnee;
    struct cellule *suivant;
} *Pile;

```



typedef Liste Pile;

REPRÉSENTATION CHAÎNÉE D'UNE PILE

P : Pile initPile()	↔	L : Liste initListe()
estVidePile(p)	↔	estVideListe(l)
Fonction estPleinPile Pile → Booléenne • Fonction estPleinPile(p) → Booléenne • P.F p : Pile (E) • Debut • Renvoyer faux • Fin		

©B. DERDOURI

CM Piles Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

27

REPRÉSENTATION CHAÎNÉE D'UNE PILE

sommetPile(p)	↔	donneeListe(l)
empiler(elt, p)	↔	inserTete(elt, l)
dépiler(p)	↔	suppTete(l)

©B. DERDOURI

CM Piles Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

28

COMPARAISONS ENTRE TABLEAUX ET LISTES CHAÎNÉES

- Dans les deux structures de données (Contigüe et Chaînée), chaque primitive ne prend que quelques opérations (complexité en temps constant).
- Par contre, la gestion de la pile par les listes chaînées présente l'énorme avantage à savoir la pile peut avoir une capacité virtuellement illimitée (limité seulement par la capacité de la mémoire centrale), la mémoire étant allouée au fur et à mesure des besoins.
- Par contre, la gestion de la pile par des tableaux, la mémoire est allouée au départ avec une capacité fixée.